Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial

Optimasi Pengisian Paket pada Truk Peti Kemas dengan Algoritma Local Search

Tahun Akademik 2026–2027



Disusun oleh:

Kelompok 37

Samuelson Dharmawan Tanuraharja – 13524001
Nicholas Wise Saragih Sumbayak – 13524037
Neswa Eka Anggara – 13524136

Laboratorium Inteligensi Artifisial
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **3**
1.1 Latar Belakang . . . 3
1.2 Rumusan Persoalan dan Tujuan . . . 3
1.3 Ruang Lingkup Persoalan . . . 3

2 Dasar Teori **5**
2.1 Optimasi dan *Local Search* . . . 5
2.2 *State*, *Neighbor*, dan *Search Landscape* . . . 5
2.3 *Hill Climbing* . . . 6
2.4 *Simulated Annealing* . . . 6
2.5 *Genetic Algorithm* . . . 7

3 Pembahasan dan Implementasi **8**
3.1 Arsitektur Aplikasi . . . 8
3.1.1 Struktur Aplikasi . . . 8
3.1.2 Penjelasan Tech Stack . . . 9
3.2 Pemodelan Problem dan Constraint . . . 10
3.2.1 Data Problem . . . 10
3.2.2 Representasi State dan Orientasi . . . 11
3.2.3 Validasi Constraint . . . 11
3.2.4 Objective Function . . . 13
3.3 Implementasi Algoritma *Local Search* . . . 14
3.3.1 Inisialisasi State dan Pembangkitan Neighbor . . . 14
3.3.2 Hill Climbing . . . 15
3.3.3 Simulated Annealing . . . 17
3.3.4 Genetic Algorithm . . . 18
3.4 Rancangan Eksperimen . . . 19
3.4.1 Data Uji dan Lingkungan Eksperimen . . . 19
3.4.2 Konfigurasi Parameter dan Pengulangan Eksperimen . . . 19
3.5 Hasil Eksperimen dan Visualisasi Program . . . 20
3.5.1 Hill Climbing . . . 20
3.5.2 Simulated Annealing . . . 20
3.5.3 Genetic Algorithm . . . 20
3.5.4 Visualisasi State Awal dan State Akhir . . . 20
3.6 Analisis Eksperimen . . . 20
3.6.1 Perbandingan Kualitas Solusi . . . 20
3.6.2 Perbandingan Durasi Pencarian . . . 20
3.6.3 Konsistensi Hasil Eksperimen . . . 20
3.6.4 Pengaruh Parameter Genetic Algorithm . . . 20

4 Kesimpulan dan Saran **21**
4.1 Kesimpulan . . . 21
4.2 Saran . . . 21

5 Pembagian Tugas Tiap Anggota Kelompok **21**

Daftar Pustaka **21**

A Form Penggunaan Generative AI **21**

B Lampiran Lainnya (Bila Ada) **21**

1 Pendahuluan

1.1 Latar Belakang

Mejiro Express ingin mengatur pengiriman paket dengan truk peti kemas. Special Week, pengemudinya, perlu segera memutuskan paket mana yang akan dibawa dan bagaimana paket itu disusun. Setiap paket punya ukuran, nilai, berat, dan sifat kerapuhan yang berbeda. Susunannya harus muat di dalam peti kemas, stabil, dan tidak melebihi kapasitas angkut [1].

Pilihan posisi dan orientasi bertambah seiring banyaknya paket. Bahkan setelah paket yang akan dibawa dipilih, masih ada banyak cara untuk menatanya. Tugas ini memakai ***local search*** (pencarian lokal) untuk mencari susunan bernilai tinggi melalui perubahan kecil pada susunan yang sudah ada.

1.2 Rumusan Persoalan dan Tujuan

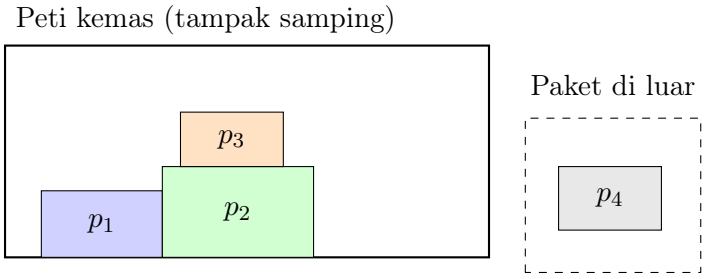
Masukannya berupa paket berbentuk balok atau kubus serta peti kemas dengan ukuran dan kapasitas angkut tertentu. Program harus memilih paket yang dimuat, lalu menentukan posisi dan orientasi tiap paket tersebut. Targetnya adalah memaksimalkan ***objective function*** (fungsi objektif), yaitu total nilai paket yang berhasil dimuat dalam susunan yang sah. Paket yang tidak dimuat tetap tercatat di luar peti kemas [1].

Pekerjaan ini menerapkan empat varian *hill climbing* (HC), *simulated annealing* (SA), dan *genetic algorithm* (GA). Program mencatat state awal, state akhir, state terbaik, skor, riwayat pencarian, dan waktu eksekusi untuk mendukung perbandingan kualitas serta durasi pencarian. Pengulangan dengan seed berbeda digunakan dalam rancangan eksperimen untuk mengamati variasi hasil pencarian acak [1].

1.3 Ruang Lingkup Persoalan

Posisi paket memakai koordinat bilangan bulat pada ruang tiga dimensi. Paket dapat diputar sebesar 90° terhadap sumbu ruang. Setiap paket yang dimuat harus berada sepenuhnya di dalam peti kemas, tidak bertumpang tindih, dan memiliki penopang. Paket rapuh tidak boleh menopang paket lain. Total berat muatan juga tidak boleh melampaui kapasitas truk. Paket yang tidak dimuat berada sepenuhnya di luar peti kemas [1].

Implementasi mendukung satu truk maupun beberapa truk dalam satu problem. Setiap paket dapat ditugaskan kepada paling banyak satu truk; nilai muatan dijumlahkan pada seluruh armada. Selain total nilai sebagai objektif utama, tersedia objektif tambahan yang membobot nilai berdasarkan ETA paket. Kedua mode memakai aturan kelayakan susunan yang sama.



Gambar 1: Ilustrasi konseptual paket termuat dan paket di luar peti kemas. Susunan sebenarnya ditentukan dalam tiga dimensi.

2 Dasar Teori

2.1 Optimasi dan *Local Search*

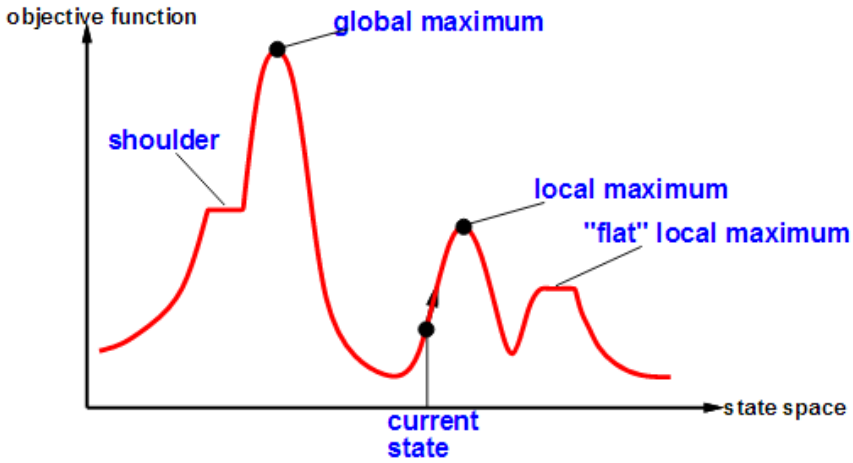
Dalam optimasi, *objective function* memberi nilai pada setiap kandidat solusi. Pencarian diarahkan ke kandidat dengan nilai setinggi mungkin. Jika ada ***constraint*** (batasan), kandidat yang dapat dipilih harus memenuhi constraint tersebut. Dengan $\mathcal{F}$ sebagai himpunan kandidat yang sah, tujuan umum masalah maksimisasi dapat ditulis $\max_{s \in \mathcal{F}} f(s)$, dengan $f(s)$ sebagai nilai kandidat s [2].

Local search bekerja pada satu kandidat lengkap dan memperbaikinya melalui perubahan bertahap. Yang menjadi hasil adalah konfigurasi akhirnya. Urutan perubahan selama pencarian tidak ikut menentukan nilai solusi. Pendekatan ini berguna ketika ruang kemungkinan terlalu besar untuk diperiksa satu per satu [2, 3].

2.2 *State*, *Neighbor*, dan *Search Landscape*

State (keadaan) adalah satu kandidat solusi yang lengkap. Pencarian dimulai dari *initial state* (state awal), yang bisa dibangkitkan secara acak. Sebuah *move* (langkah perubahan) menghasilkan ***neighbor*** (tetangga), yaitu state yang berbeda dalam sebagian kecil komponennya. Kumpulan neighbor yang dapat dicapai menentukan ke mana pencarian bisa bergerak dari state saat ini [3].

Nilai setiap state dapat dibayangkan sebagai ketinggian dalam *search landscape* (lanskap pencarian). ***Local maximum*** (puncak lokal) bernilai setidaknya sama tinggi dengan semua neighbor-nya, meskipun mungkin ada state lain yang nilainya lebih tinggi. Nilai tertinggi di seluruh ruang pencarian disebut *global maximum* (puncak global). Ada pula *plateau* (daerah datar), yaitu kumpulan state bertetangga dengan nilai yang sama. Local maximum dan plateau menjelaskan mengapa pencarian yang selalu mengejar kenaikan nilai bisa berhenti terlalu cepat [2, 3].



Gambar 2: *Search landscape* dengan global maximum, local maximum, dan plateau. Diambil dari salindia *Beyond Classical Search* [3].

2.3 *Hill Climbing*

Seperti terlihat pada Gambar 2, pencarian yang hanya memilih kenaikan nilai dapat berhenti di local maximum. ***Hill climbing*** membandingkan neighbor dengan state saat ini. Dalam *steepest-ascent hill climbing*, seluruh neighbor dievaluasi, lalu dipilih yang bernilai tertinggi jika lebih baik daripada state saat ini. *Stochastic hill climbing* memilih secara acak dari neighbor yang meningkatkan nilai. Peluang pemilihannya dapat dibuat seragam atau dibobot berdasarkan besar peningkatan. Kedua varian menolak move yang tidak meningkatkan skor sehingga dapat berhenti di local maximum atau plateau [2, 3].

Hill climbing with sideways move menerima neighbor dengan skor sama untuk bergerak di plateau. Jumlah move setara biasanya dibatasi agar pencarian tidak terus berputar. *Random-restart hill climbing* mengulang pencarian dari initial state berbeda dan menyimpan solusi terbaik di antara pengulangan. Restart memperluas bagian search landscape yang dijelajahi, tetapi jumlah restart terbatas tidak menjamin optimum global [2, 3].

2.4 *Simulated Annealing*

Hill climbing tidak selalu bisa keluar dari local maximum dengan move yang hanya memperbaiki nilai. ***Simulated annealing*** sesekali menerima move yang menurunkan nilai agar pencarian dapat berpindah ke daerah lain. Peluang menerima penurunan ditentukan oleh *temperature* (temperatur). Untuk $\Delta f = f(s') - f(s) < 0$ dan $T > 0$, peluang menerima neighbor s' adalah

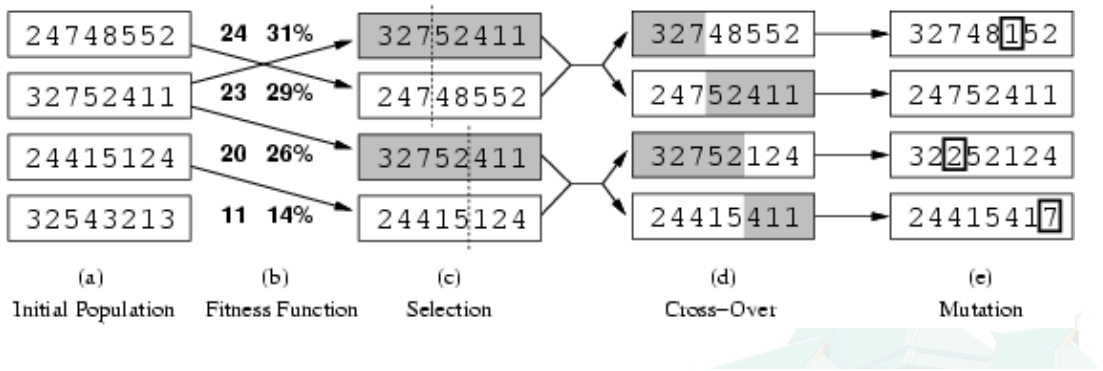
$$\Pr(\text{accept } s') = \exp\left(\frac{\Delta f}{T}\right). \tag{1}$$

Pada temperature tinggi, algoritma lebih sering menerima penurunan nilai sehingga dapat keluar dari local maximum. Seiring temperature turun, peluang itu mengecil. *Cooling schedule* (jadwal penurunan temperatur) menentukan seberapa cepat perilaku pencarian berubah. Jadwal yang terlalu cepat dapat membuat pencarian kembali terjebak sebelum sempat menjelajahi state lain [2, 3].

2.5 *Genetic Algorithm*

Jika hill climbing dan simulated annealing mengubah satu state dalam satu lintasan pencarian, ***genetic algorithm*** bekerja dengan *population* (populasi) yang berisi sejumlah *individual* (kandidat solusi). Setiap individual dinilai memakai *fitness function* (fungsi penilaian). Melalui *selection* (pemilihan induk), individual yang lebih baik mendapat peluang lebih besar untuk menghasilkan keturunan [2, 3].

Generasi baru dibentuk melalui *crossover* (penggabungan bagian dari dua induk) dan *mutation* (perubahan acak pada individual). Crossover mewariskan bagian dari solusi yang sudah ada, sementara mutation membantu menjaga keragaman population. Ukuran population dan banyaknya *generation* (generasi) memengaruhi jumlah kandidat yang diperiksa serta waktu pencarian. Bila suatu operator menghasilkan kandidat yang melanggar constraint, kandidat itu perlu ditangani sebelum dipakai sebagai solusi [2, 3].



Gambar 3: Alur selection, crossover, dan mutation pada genetic algorithm. Diambil dari salindia *Beyond Classical Search* [3].

3 Pembahasan dan Implementasi

3.1 Arsitektur Aplikasi

Pengguna mengubah paket dan truk melalui formulir web yang dimulai dari fixture armada bawaan di `src/frontend/src/mocks/fleetFixture.ts`. Belum ada alur impor berkas problem.

Konfigurasi diperiksa melalui endpoint `POST /api/problems/validate`. Endpoint ini memvalidasi model dan batas masukan, bukan susunan paket. Ketika run dimulai, frontend memeriksa parameter lalu mengirim problem, algoritma, seed, objektif, parameter, dan pilihan riwayat ke `POST /api/experiments/run`. Backend memvalidasi skema dan anggaran eksekusi, membangkitkan state awal, menjalankan algoritma, lalu memeriksa kelayakan state awal, akhir, dan terbaik serta kecocokan skornya sebelum membentuk respons.

Run dari UI memakai `persist=false`. Hasil yang disimpan dapat dimuat kembali melalui `GET/api/experiments`. Endpoint preset GA dan batch tersedia melalui `GET/api/experiments/presets/genetic_algorithm` dan `POST/api/experiments/batch`. Problem yang sedang diedit untuk run berikutnya terpisah dari problem asli milik run yang sedang ditampilkan.

3.1.1 Struktur Aplikasi

Folder utama aplikasi tersusun sebagai berikut. Pohon ini hanya menampilkan direktori yang berperan dalam alur pencarian dan visualisasi.

```
.
|-- data/
|-- output/
|-- src/
    |-- backend/
    | |-- app/
    | |-- api/
    | |-- domain/
    | |-- experiments/
    | |-- optimization/
    |-- frontend/
        |-- src/
            |-- api/
            |-- components/
            |-- stores/
```

Di backend, `domain` mendefinisikan data, `optimization` memuat validator dan algoritma, sedangkan `experiments` mengatur eksekusi, ringkasan, dan pencatatan hasil. Di frontend, `components` memuat form, grafik, inspector, dan tampilan truk; `api` menangani request serta pemeriksaan respons, dan `stores` menyimpan run terpilih serta indeks riwayat. Fixture awal berada di `src/frontend/src/mocks`; `data/` belum memuat dataset. Jika persistensi diaktifkan, run tunggal disimpan sebagai JSON di `output/experiments/`, sedangkan batch menyimpan JSON dan CSV di `output/batches/`. Berkas-berkas tersebut merupakan keluaran yang dapat dibuat, bukan hasil yang telah tersedia di repositori saat ini.

3.1.2 Penjelasan Tech Stack

Backend ditulis dengan **Python 3.12**. **FastAPI** menyediakan endpoint untuk memvalidasi problem, menjalankan eksperimen tunggal atau batch, dan membaca hasil yang tersimpan. Model **Pydantic** memeriksa tipe serta rentang nilai saat request diterima. Sesudah itu, modul optimasi menjalankan aturan yang sama untuk semua algoritma. **NumPy** digunakan untuk occupancy grid tiga dimensi, sehingga pemeriksaan ruang terisi dapat dilakukan pada irisan sel paket.

Frontend dibangun dengan **React** dan **TypeScript**. Komponen React memisahkan form problem, kontrol pencarian, visualisasi, dan ringkasan hasil. TypeScript membantu menjaga bentuk request dan respons tetap sesuai kontrak backend. **Vite** dipakai untuk menjalankan frontend selama pengembangan dan menghasilkan berkas aplikasi saat build.

Untuk menampilkan hasil, **React Three Fiber** dan **Three.js** menggambar peti kemas serta posisi paket dalam ruang tiga dimensi. **Recharts** menampilkan perubahan skor selama iterasi. **Zustand** menyimpan eksperimen yang dipilih dan indeks iterasi untuk replay.

3.2 Pemodelan Problem dan Constraint

Pemodelan dimulai dari data paket dan truk. Dari data itu, program membentuk state yang mencatat posisi setiap paket. State lalu melewati pemeriksaan constraint sebelum objective dihitung. Urutan ini dipakai bersama oleh seluruh algoritma, sehingga nilai yang dibandingkan selalu berasal dari **susunan yang sah**.

3.2.1 Data Problem

Kelas `Problem` menyimpan dua daftar: `PackageSpec` dan `TruckSpec`. Setiap paket memiliki ID, dimensi lebar, panjang, dan tinggi, kemudian `value`, `weight`, `is_fragile`, serta `eta`. Truk menyimpan ID, tiga dimensi ruang muat, dan kapasitas maksimum. Cuplikan 1 hingga 3 memperlihatkan hubungan ketiga kelas tersebut.

Cuplikan 1: Atribut paket pada `PackageSpec`

```
class PackageSpec(BaseModel):
    model_config = ConfigDict(frozen=True, allow_inf_nan=False)

    id: str
    width: int = Field(gt=0)
    length: int = Field(gt=0)
    height: int = Field(gt=0)
    value: float = Field(ge=0)
    weight: float = Field(ge=0)
    is_fragile: bool
    eta: int = Field(ge=0)
```

Cuplikan 2: Atribut truk pada `TruckSpec`

```
class TruckSpec(BaseModel):
    model_config = ConfigDict(frozen=True, allow_inf_nan=False)

    id: str
    width: int = Field(gt=0)
    length: int = Field(gt=0)
    height: int = Field(gt=0)
    max_capacity: float = Field(ge=0)
```

Cuplikan 3: Daftar paket dan truk dalam `Problem`

```
class Problem(BaseModel):
    packages: list[PackageSpec]
    trucks: list[TruckSpec]

    @model_validator(mode="after")
    def validate_ids_and_trucks(self) -> "Problem":
        if not self.trucks:
            raise ValueError("at least one truck is required")
        if len({package.id for package in self.packages}) != len(self.packages):
            raise ValueError("package IDs must be unique")
        if len({truck.id for truck in self.trucks}) != len(self.trucks):
            raise ValueError("truck IDs must be unique")
        return self
```

Validasi masukan dilakukan sejak model dibuat. Dimensi harus berupa bilangan bulat positif, sedangkan nilai, berat, ETA, dan kapasitas tidak boleh negatif maupun tak hingga.

`Problem` menolak ID paket atau truk yang berulang dan mensyaratkan setidaknya satu truk. Karena `trucks` berupa daftar, struktur yang sama dapat dipakai untuk satu truk maupun beberapa truk. Setelah data ini tersedia, penempatan setiap paket dicatat dalam state.

3.2.2 Representasi State dan Orientasi

Kelas `State` berisi daftar `PackagePlacement`. Pada state yang sah, **setiap paket memiliki tepat satu placement**. Di dalamnya tersimpan ID paket dan truk, `Position`, serta `Orientation`. Koordinat `Position` menunjuk sudut kiri-bawah-belakang paket. Sumbu x mengikuti lebar truk, y mengikuti panjang, dan z mengikuti tinggi. Paket di luar truk ditandai dengan `truck_id` dan `position` yang sama-sama bernilai `None`. Keduanya harus terisi bersama ketika paket dimuat, sebagaimana diperiksa oleh `validate_assignment` pada Cuplikan 4.

Cuplikan 4: Model penempatan paket

```
class PackagePlacement(BaseModel):
    package_id: str
    truck_id: str | None = None
    position: Position | None = None
    orientation: Orientation = Orientation.WLH

    @model_validator(mode="after")
    def validate_assignment(self) -> "PackagePlacement":
        if (self.truck_id is None) != (self.position is None):
            raise ValueError("truck_id and position must both be set or both be None")
        return self
```

Enam pilihan `Orientation` didefinisikan pada `app/domain/package.py`. Fungsi `get_oriented_dimensions` mengubah orientasi menjadi dimensi efektif (w_i, l_i, h_i) tanpa mengubah dimensi asli paket. Untuk kubus atau balok dengan sisi yang sama, beberapa orientasi menghasilkan dimensi efektif identik. Fungsi `get_unique_orientations` menghapus pilihan duplikat tersebut saat orientasi akan diacak. State yang terbentuk kemudian diperiksa sebagai satu kesatuan, bukan per paket saja.

3.2.3 Validasi Constraint

Fungsi `validate_state` pada `app/optimization/constraints.py` menjadi **pemeriksa akhir setiap kandidat**. Pemeriksaan dimulai dari kelengkapan assignment. Setiap paket dalam `Problem` harus muncul sekali dalam `State`, sedangkan ID paket dan truk yang tidak dikenal ditolak. Paket yang berada di luar tidak memakai ruang maupun kapasitas truk.

Validator membuat *occupancy grid* (grid keterisian) NumPy bagi setiap truk. Bentuk grid mengikuti (W_t, L_t, H_t), dengan sel kosong bernilai -1 dan sel terisi menyimpan indeks paket. Jika paket i ditempatkan di truk t pada (x_i, y_i, z_i) dengan dimensi efektif (w_i, l_i, h_i), seluruh bagian paket harus memenuhi

$$\begin{aligned} 0 \le x_i, \quad x_i + w_i \le W_t, \\ 0 \le y_i, \quad y_i + l_i \le L_t, \\ 0 \le z_i, \quad z_i + h_i \le H_t. \end{aligned} \tag{2}$$

Dalam kode, `_collect_occupancies` membandingkan posisi dan dimensi paket dengan ukuran grid. Cuplikan 5 menunjukkan pemeriksaan paket yang melewati dinding peti

kemas. Setelah lolos, irisan sel yang akan ditempati diperiksa lagi. Cuplikan 6 menolak paket ketika irisan tersebut sudah berisi indeks paket lain. Karena grid dibuat per truk, paket pada truk berbeda tidak saling bertabrakan.

Cuplikan 5: Pemeriksaan batas peti kemas

```
if any(
    start < 0 or start + size > limit
    for start, size, limit in zip(origin, dimensions, grid.shape, strict=True)
):
    errors.append(
        f"Package {placement.package_id} is out of bounds in truck {truck_id}"
    )
    unsafe_trucks.add(truck_id)
    continue
```

Cuplikan 6: Pemeriksaan overlap pada occupancy grid

```
for placement, package_index, region in regions:
    truck_id = placement.truck_id
    assert truck_id is not None
    cells = occupancies[truck_id][region]
    if np.any(cells != -1):
        errors.append(
            f"Package {placement.package_id} overlaps another in truck {truck_id}"
        )
        unsafe_trucks.add(truck_id)
        continue
    cells[...] = package_index
```

Paket pada $z_i = 0$ ditopang oleh lantai truk. Untuk paket di atas lantai, validator memeriksa sel tepat di bawah alasnya, yakni pada $z_i - 1$. Paket dinyatakan **melayang** jika tidak ada satu pun sel alas yang mendapat support dari paket lain. Jika ada support tetapi salah satu paket penopang bersifat rapuh, susunan juga ditolak. Pemeriksaan ini mengikuti definisi support pada tugas, yaitu cukup ada satu titik alas yang ditopang. Validator tidak mensyaratkan seluruh alas paket bersentuhan dengan penopang. Bagian inti pemeriksaan tersebut tampak pada Cuplikan 7.

Cuplikan 7: Pemeriksaan support dan paket rapuh

```
w, l, _ = get_oriented_dimensions(
    packages[placement.package_id], placement.orientation
)
x, y, z = position.x, position.y, position.z
below = occupancies[truck_id][x : x + w, y : y + l, z - 1]
supporters = np.unique(below[below >= 0])
if supporters.size == 0:
    support_errors.append(
        f"Package {placement.package_id} is floating in truck {truck_id}"
    )
elif any(problem.packages[int(index)].is_fragile for index in supporters):
    fragile_errors.append(
        f"Package {placement.package_id} rests on fragile support "
        f"in truck {truck_id}"
    )
```

Setelah posisi dan support diperiksa, validator menghitung beban setiap truk. Untuk

truk t, jumlah berat paket yang ditugaskan kepadanya harus memenuhi

$$\sum_{i \in P_t} \text{weight}_i \leq \text{max_capacity}_t. \tag{3}$$

Cuplikan 8 menunjukkan bahwa berat dijumlahkan dengan `Decimal`. Dengan cara ini, batas seperti $0.1 + 0.2 \leq 0.3$ tidak keliru akibat pembulatan bilangan *floating point*. Hasil `validate_state` berupa `ValidationResult` yang memuat status `valid`, daftar `errors`, dan total berat per truk. Jika assignment atau geometri sudah ambigu, pemeriksaan support dilewati karena grid tidak lagi dapat dipercaya, sementara kesalahan yang sudah ditemukan tetap dilaporkan.

Cuplikan 8: Penjumlahan berat dan pemeriksaan kapasitas

```
exact_weights = {truck.id: Decimal(0) for truck in problem.trucks}
for placement in loaded:
    assert placement.truck_id is not None
    exact_weights[placement.truck_id] += Decimal(
        str(packages[placement.package_id].weight)
    )
truck_weights = {
    truck_id: float(weight) for truck_id, weight in exact_weights.items()
}
for truck in problem.trucks:
    if exact_weights[truck.id] > Decimal(str(truck.max_capacity)):
        errors.append(
            f"Truck {truck.id} exceeds capacity: "
            f"{truck_weights[truck.id]} > {truck.max_capacity}"
        )
```

3.2.4 Objective Function

Setelah state dinyatakan sah, program menghitung nilainya melalui `ObjectiveMode` pada `app/optimization/objective.py`. **Mode utama**, `TOTAL_VALUE`, menjumlahkan `value` seluruh paket yang dimuat pada armada:

$$f_{\text{total}}(s) = \sum_{i:\, \text{truck}_i \neq \text{None}} \text{value}_i. \tag{4}$$

Mode tambahan, `ETA_WEIGHTED_VALUE`, memberi bobot lebih besar pada paket yang ETA-nya lebih dekat tanpa menambah atribut masukan:

$$f_{\text{ETA}}(s) = \sum_{i:\, \text{truck}_i \neq \text{None}} \frac{\text{value}_i}{\text{eta}_i + 1}. \tag{5}$$

Kontribusi setiap paket termuat dihitung oleh `objective_contributions`. Hasilnya dijumlahkan oleh `evaluate_objective` dengan `math.fsum`. Evaluator tidak memberikan penalti bagi pelanggaran constraint dan tidak memeriksa kelayakan state. Pemanggil harus memakai `validate_state` terlebih dahulu. Dengan begitu, **skor hanya dibandingkan di antara state yang sah**.

3.3 Implementasi Algoritma *Local Search*

3.3.1 Inisialisasi State dan Pembangkitan Neighbor

Sebelum pencarian dimulai, `runner.py` membuat satu *random number generator* dari *seed* yang diterima. Generator yang sama diteruskan ke inisialisasi dan algoritma agar run dengan input serta seed yang sama dapat diulang. Hill climbing dan simulated annealing menerima satu initial state dari `generate_random_feasible_state`. Genetic algorithm memakai fungsi yang sama untuk membentuk anggota populasi awal.

Inisialisasi memulai semua paket di luar truk, lalu mengacak urutan paket dan tujuan penempatannya. Untuk setiap truk yang dicoba, fungsi memilih orientasi dan posisi. Koordinat x dan y diambil dari rentang yang masih muat, sedangkan z diambil dari lantai atau permukaan atas paket lain. Kandidat baru hanya disimpan bila lolos `validate_state`. Paket dapat tetap di luar saat target luar terpilih atau ketika percobaan penempatan gagal. Cuplikan 9 menunjukkan titik awal konstruksi tersebut.

Cuplikan 9: Pembentukan initial state dalam `initializer.py`

```
state = State(
    placements=[PackagePlacement(package_id=p.id) for p in problem.packages]
)
indices = list(range(len(problem.packages)))
rng.shuffle(indices)
```

Setelah initial state tersedia, neighbor dibentuk lewat tiga move pada `neighbors.py`. `RELOCATE` memindahkan paket ke koordinat atau truk lain, termasuk ke luar truk. `ROTATE` mengubah orientasi paket termuat sambil mempertahankan titik posisinya. `SWAP` menukar tujuan truk dan posisi dua paket, sementara masing-masing mempertahankan orientasinya. Peluang pemilihan awalnya berturut-turut 0,50, 0,25, dan 0,25. Setiap move bekerja pada salinan state agar state asal tidak berubah.

Proposal move belum tentu memenuhi constraint. Untuk hill climbing dan simulated annealing, `generate_random_neighbor` mengulang proposal sampai menemukan state sah atau mencapai batas percobaan, seperti pada Cuplikan 10. Hill climbing dapat meminta beberapa neighbor berbeda melalui `sample_valid_neighbors`, sedangkan simulated annealing mengambil satu. Pada genetic algorithm, proposal mutasi yang belum sah boleh masuk ke tahap repair sebelum dinilai.

Cuplikan 10: Penerimaan neighbor yang sah dalam `neighbors.py`

```
for _ in range(config.max_attempts):
    candidate = propose_random_move(problem, state, rng, config)
    if candidate is not None and validate_state(problem, candidate.state).valid:
        return candidate
return None
```

3.3.2 Hill Climbing

Keempat varian hill climbing diimplementasikan dalam kelas `HillClimbing` pada `app/optimization/algorithms/hill_climbing.py`. Setiap iterasi mengambil sampel neighbor sah melalui `sample_valid_neighbors`, kemudian menghitung objective-nya. Jumlah sampel diatur oleh `candidates_per_iteration`, dengan nilai default 50. Pembangkitan dibatasi hingga jumlah kandidat dikali `max_attempts` percobaan, dengan default 30, sehingga sampel yang diperoleh dapat lebih sedikit dari permintaan. Perbedaan tiap varian terletak pada cara memilih kandidat dan melanjutkan pencarian. Cuplikan berikut merangkum bagian implementasi yang digunakan masing-masing varian, dengan pencatatan riwayat tidak ditampilkan.

Steepest-ascent hill climbing memilih neighbor dengan skor tertinggi dari sampel pada setiap iterasi. Jika beberapa kandidat memiliki skor tertinggi yang sama, salah satunya dipilih secara acak. Perpindahan hanya dilakukan apabila skor tersebut lebih tinggi daripada skor state saat ini, seperti dirangkum pada Cuplikan 11. Ketika tidak ada peningkatan dalam sampel, pencarian berhenti dengan alasan `local_optimum`. Karena pemeriksaan memakai sampel terbatas, kondisi ini belum membuktikan bahwa seluruh neighborhood tidak memiliki solusi yang lebih baik.

Cuplikan 11: Inti pemilihan neighbor pada steepest-ascent

```
top_score = max(scores)
if top_score <= current_score:
    termination_reason = "local_optimum"
    break
pool = [
    (n, score)
    for n, score in zip(neighbors, scores)
    if score == top_score
]
selected, selected_score = rng.choice(pool)
current, current_score = selected.state, selected_score
```

Stochastic hill climbing mengumpulkan semua neighbor sampel yang skornya lebih tinggi daripada state saat ini, lalu memilih satu secara acak dengan peluang seragam. Kandidat yang terpilih dapat memiliki skor lebih rendah daripada kandidat terbaik dalam sampel, selama tetap memperbaiki state saat ini. Cuplikan 12 memperlihatkan penyaringan kandidat peningkat dan pemilihannya. Jika kumpulan tersebut kosong, pencarian berhenti dengan alasan `local_optimum`; jumlah kandidat peningkat yang ditemukan juga dicatat dalam riwayat.

Cuplikan 12: Pemilihan acak dari neighbor peningkat pada stochastic

```
if is_stochastic:
    pool = [
        (n, score)
        for n, score in zip(neighbors, scores)
        if score > current_score
    ]
    if not pool:
        termination_reason = "local_optimum"
        break
    selected, selected_score = rng.choice(pool)
    current, current_score = selected.state, selected_score
```

Hill climbing with sideways move memilih kandidat terbaik seperti steepest-ascent, tetapi juga mengizinkan perpindahan dengan skor yang sama untuk menelusuri plateau. Perpindahan setara dibatasi oleh `max_sideways`, yang secara default bernilai 100 kali berturut-turut. Setiap peningkatan skor mengembalikan hitungan tersebut ke nol, sedangkan perpindahan setara menambahnya satu, sebagaimana ditunjukkan pada Cuplikan 13. Jika kandidat terbaik tetap setara setelah batas positif ini tercapai, pencarian berhenti dengan alasan `max_sideways`. Saat batas bernilai nol, kandidat setara langsung ditolak dengan alasan `local_optimum`. Move yang menurunkan skor selalu ditolak, dan jumlah sideways move disimpan dalam hasil pencarian.

Cuplikan 13: Penerimaan dan penghitungan sideways move

```
if top_score > current_score or (
    top_score == current_score and consecutive_sideways < max_sideways
):
    pool = [
        (n, score)
        for n, score in zip(neighbors, scores)
        if score == top_score
    ]
    selected, selected_score = rng.choice(pool)
    improved = selected_score > current_score
    current, current_score = selected.state, selected_score
    if improved:
        consecutive_sideways = 0
    else:
        consecutive_sideways += 1
        sideways_moves += 1
```

Random-restart hill climbing menjalankan steepest-ascent beberapa kali dari state awal yang berbeda. Climb pertama memakai initial state dari runner; setiap restart berikutnya membangkitkan state sah baru melalui `generate_random_feasible_state`. Dengan default `max_restarts` sebesar 10, algoritma menjalankan 11 climb, masing-masing dengan batas `max_iterations` sendiri. Cuplikan 14 menunjukkan pengulangan climb dan pemilihan hasil terbaiknya. State akhir diambil dari climb terakhir, sedangkan state terbaik diambil dari seluruh climb. Riwayat digabung dengan indeks iterasi berurutan, penanda restart, dan skor terbaik kumulatif; hasil juga mencatat jumlah iterasi serta skor terbaik tiap climb. Setelah semua restart selesai, alasan terminasi keseluruhan adalah `max_restarts`.

Cuplikan 14: Pengulangan steepest-ascent pada random restart

```
start = initial_state
for restart in range(config.max_restarts + 1):
    if restart > 0:
        start = generate_random_feasible_state(problem, rng)
    climb = self._climb(
        problem,
        start,
        variant=HillClimbingVariant.STEEPEST_ASCENT,
        objective_mode=objective_mode,
        rng=rng,
        config=config,
        include_state_history=include_state_history,
    )
    climbs.append(climb)
```

```
best = max(climbs, key=lambda climb: climb.best_score)
```

Setiap climb juga berhenti jika tidak ada neighbor sah (`no_valid_neighbor`) atau batas iterasi tercapai (`max_iterations`). State saat ini dan state terbaik disimpan terpisah. Melalui API, pencarian dibatasi hingga 2.000 iterasi per climb, 50 kandidat per iterasi, dan 20 restart.

3.3.3 Simulated Annealing

Kelas `SimulatedAnnealing` memulai pencarian dari state sah yang sama jenisnya dengan hill climbing, tetapi hanya mengambil **satu neighbor acak** pada setiap iterasi. Move dengan $\Delta f \geq 0$ langsung diterima. Untuk $\Delta f < 0$, algoritma menggunakan peluang $\exp(\Delta f/T)$ sebagaimana dijelaskan pada Bab 2. Cuplikan 15 menunjukkan keputusan tersebut di dalam kode.

Cuplikan 15: Penerimaan move pada `simulated_annealing.py`

```
delta = candidate_score - current_score
acceptance_probability = (
    1.0 if delta >= 0 else math.exp(delta / temperature)
)
accepted = delta >= 0 or rng.random() < acceptance_probability
if accepted:
    current = candidate.state.model_copy(deep=True)
    current_score = candidate_score
```

Temperatur awal dapat diberikan sebagai parameter. Jika kosong, nilainya diambil dari rata-rata value paket dengan batas bawah 1. Setelah satu iterasi selesai, temperatur dikalikan `alpha`, yang bernilai antara nol dan satu. Pencarian berhenti saat temperatur berada di bawah `min_temperature`, batas iterasi tercapai, atau neighbor sah tidak ditemukan. `best_state` disimpan terpisah dari state saat ini, sebab simulated annealing dapat menerima move yang menurunkan skor. Riwayat iterasi mencatat temperatur, peluang penerimaan, keputusan move, dan skor. Parameter `stuck_patience` hanya menandai dan menghitung kejadian ketika skor terbaik tidak membaik cukup lama; parameter ini tidak menghentikan pencarian.

3.3.4 Genetic Algorithm

Kelas `GeneticAlgorithm` memulai pencarian dengan **populasi state sah**. Runner tidak memberikan initial state tunggal; algoritma membangkitkan populasi awal, lalu champion populasi tersebut dicatat sebagai `initial_state` dan `initial_score` pada hasil. Satu state menjadi satu individual dan objective-nya menjadi fitness. Pemilihan induk memakai `roulette_select`, sehingga fitness lebih tinggi mendapat peluang lebih besar. Jika semua fitness nol, induk dipilih seragam secara acak. `uniform_crossover` menyusun anak dengan mengambil placement setiap paket dari salah satu dari dua induk, masing-masing dengan peluang setengah.

Dalam setiap generasi, satu individual dengan fitness tertinggi disalin langsung sebagai *elite*. Individual lain dibentuk melalui selection, crossover sesuai `crossover_probability`, dan mutation sesuai `mutation_probability`. Mutation memilih salah satu dari relocate, rotate, atau swap. Sebelum masuk ke populasi berikutnya, anak diproses oleh `repair_state`, seperti terlihat pada Cuplikan 16.

Cuplikan 16: Pembentukan generasi baru pada `genetic_algorithm.py`

```
next_population = [champion.model_copy(deep=True)]
while len(next_population) < config.population_size:
    parent_a = roulette_select(population, fitnesses, rng)
    parent_b = roulette_select(population, fitnesses, rng)
    if rng.random() < config.crossover_probability:
        child = uniform_crossover(problem, parent_a, parent_b, rng)
    else:
        child = rng.choice((parent_a, parent_b)).model_copy(deep=True)
    if rng.random() < config.mutation_probability:
        proposal = rng.choice(
            (propose_relocate, propose_rotate, propose_swap)
        )(problem, child, rng)
        if proposal is not None:
            child = proposal.state
    next_population.append(repair_state(problem, child, rng))
```

Repair memulai dari state dengan semua paket di luar, lalu memproses paket termuat menurut koordinat z dan ID. Placement yang masih layak dipertahankan; paket yang gagal ditempatkan ulang dicoba pada orientasi, posisi, dan truk lain, dan tetap di luar bila tidak ada penempatan sah. Setiap generasi mencatat maksimum dan rata-rata fitness populasi; riwayat dimulai pada generasi 0, sehingga memiliki $G + 1$ catatan untuk G generasi. Champion generasi terakhir menjadi state akhir, sementara state terbaik sepanjang run disimpan terpisah. Elitisme mempertahankan satu champion. Skema memakai default populasi 50, 100 generasi, peluang crossover 0,80, peluang mutasi 0,20, dan satu elite. Batas API ialah populasi 100 dan 200 generasi.

3.4 Rancangan Eksperimen

3.4.1 Data Uji dan Lingkungan Eksperimen

3.4.2 Konfigurasi Parameter dan Pengulangan Eksperimen

3.5 Hasil Eksperimen dan Visualisasi Program

3.5.1 Hill Climbing

3.5.2 Simulated Annealing

3.5.3 Genetic Algorithm

3.5.4 Visualisasi State Awal dan State Akhir

3.6 Analisis Eksperimen

3.6.1 Perbandingan Kualitas Solusi

3.6.2 Perbandingan Durasi Pencarian

3.6.3 Konsistensi Hasil Eksperimen

3.6.4 Pengaruh Parameter Genetic Algorithm

4 Kesimpulan dan Saran

4.1 Kesimpulan

4.2 Saran

5 Pembagian Tugas Tiap Anggota Kelompok

Nama Lengkap	NIM	Tugas/Kontribusi
Samuelson Dharmawan Tanuraharja	13524001	
Nicholas Wise Saragih Sumbayak	13524037	
Neswa Eka Anggara	13524136	

Tabel 1: Pembagian tugas anggota kelompok

Daftar Pustaka

[1] Tim Asisten Laboratorium Inteligensi Artifisial, "Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027," ver. 1.4, Institut Teknologi Bandung, Bandung, Indonesia, Sep. 2026.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] Tim Pengajar IF3170 Inteligensi Artifisial, "Beyond Classical Search," lecture slides, Institut Teknologi Bandung, Bandung, Indonesia, 2026.

A Form Penggunaan Generative AI

B Lampiran Lainnya (Bila Ada)